

PATHCLEAR

Master Product + Technical Blueprint

Condensed from the full PathClear design conversation and final corrected decisions

"PathClear does not find the shortest path. It finds the path that your body can actually use."

"Normal navigation stops at the building. PathClear gets you to the usable entrance."

Final decisions only. Earlier alternatives that were later replaced are intentionally omitted.

Contents

- 1. Core problem and product thesis
- 2. Final product solution and user types
- 3. Accessibility routing and multimodal journey planning
- 4. Frontend product and page-by-page experience
- 5. Hackathon-winning interactions and judging story
- 6. Data ingestion, storage, caching and mock strategy
- 7. Final backend architecture
- 8. Neo4j + GDS routing graph
- 9. Supabase role and repository pattern
- 10. Agentic system with LangGraph
- 11. Hybrid GraphRAG / vector RAG
- 12. Vision, blind navigation and haptics
- 13. Backend folder structure and coding rules
- 14. Core APIs, data contracts and tables
- 15. Two-day build order and demo flow
- 16. Non-negotiable architecture rules

1. Core problem and product thesis

People with disabilities can receive a valid route from a normal navigation app and still be unable to complete the journey. The map may send a wheelchair user to stairs or an unsafe slope, a blind user to a blocked footpath, a deaf user through a high-risk crossing without strong visual alerts, or any user to the middle of a building instead of its usable entrance. In dense cities such as Mumbai, the problem is worse because footpaths can disappear, construction changes daily, parked bikes and stalls block paths, drains or broken surfaces appear, and transit stations can become inaccessible when an elevator fails. PathClear solves the gap between a route existing on a map and a person actually being able to use that route.

The product is best described as an accessibility intelligence layer over maps and transportation. It combines a user accessibility profile, an accessibility-aware graph, live/temporary hazards, multimodal transport, station pathways, computer vision, voice, haptics and entrance-level guidance into one journey.

Normal maps:

Where is the destination? -> shortest / fastest route

PathClear:

Who is the user? -> what can they physically use? -> what is happening now? -> what journey is feasible? -> what entrance can they actually enter?

Core product promise

- Never treat accessibility as a cosmetic filter added after routing.
- Do not hide uncertainty. Accessibility status and confidence are separate.
- Plan the full journey, not only the walking line.
- React when reality changes.
- Do not stop at the building coordinate; finish at the usable entrance.

2. Final product solution and user types

Accessibility profile

PathClear models accessibility as independent constraints rather than only three fixed personas. The onboarding can start with Mobility, Vision, Hearing or Multiple Needs, then collect the actual limits used by the routing system.

Dimension	Examples of stored constraints
Mobility	wheelchair/scooter, step-free required, maximum slope, surface preference, shared-road tolerance
Vision	blind/low vision, tactile preference, audible crossing preference, camera guidance, voice guidance
Hearing	deaf/hard of hearing, high-contrast alerts, visual hazard cues, haptic guidance
Combined	any combination of mobility + vision + hearing constraints

The complete product flow

OPEN APP

-> accessibility profile

-> destination

-> journey options

-> accessibility route audit

-> start journey

-> GPS/voice/haptic guidance

-> hazard or camera intervention

-> reroute if required

-> transit/station pathway guidance

-> final approach

-> last 50 feet

-> accessible entrance

-> arrival / verification feedback

3. Accessibility routing and multimodal journey planning

Accessibility-aware route calculation

The route engine works on a graph whose edges carry accessibility attributes such as length, slope, surface, stairs, ramp, tactile paving, crossing type, width, hazard status and confidence. Hard constraints make an edge unusable. Soft constraints increase its cost.

Example	Routing treatment
Wheelchair + stairs	Hard block
Wheelchair + slope above user limit	Hard block or very high cost
Wheelchair + rough surface	Penalty
Blind user + tactile paving	Preference / lower cost
Unsafe crossing	High penalty
Active construction blocking path	Block or very high penalty
Unknown accessibility data	Lower confidence and optional penalty

Route audit / nutrition label

PATHCLEAR ROUTE
Distance 1.2 km
Estimated time 16 min
STEP-FREE PASS
MAX SLOPE 3.2% PASS
SURFACE 100% PAVED
TACTILE COVERAGE 80%
ACCESSIBLE CROSSINGS 2
KNOWN HAZARDS 1
CONFIDENCE 87%
RFI 0.23

RFI is the Route Friction Index, a deterministic normalized product metric combining route difficulty signals such as slope, surface, crossings, hazards and missing data. It is not presented as a scientifically universal medical standard.

Multimodal + cost-aware journey planning

PathClear plans the complete trip and can surface walking, Metro/transit, cab, bike, auto-rickshaw and shared rickshaw segments. Each transport option is normalized into a common format with ETA, travel time, estimated cost, pickup/drop-off and accessibility implications. The user can prioritize Accessibility, Time or Cost, but accessibility hard constraints remain mandatory.

Example journey:
Walk -> Shared Rickshaw -> Metro -> Walk -> Gate 3

Each segment contains:
mode, origin, destination, duration, estimated_cost, accessibility, confidence, transfer_requirements

PathClear can present "Best accessibility", "Lowest cost" and "Fastest" journey options only among journeys that remain feasible for the user. A cheap route that violates a hard accessibility constraint is not considered valid.

GTFS + GTFS-Pathways

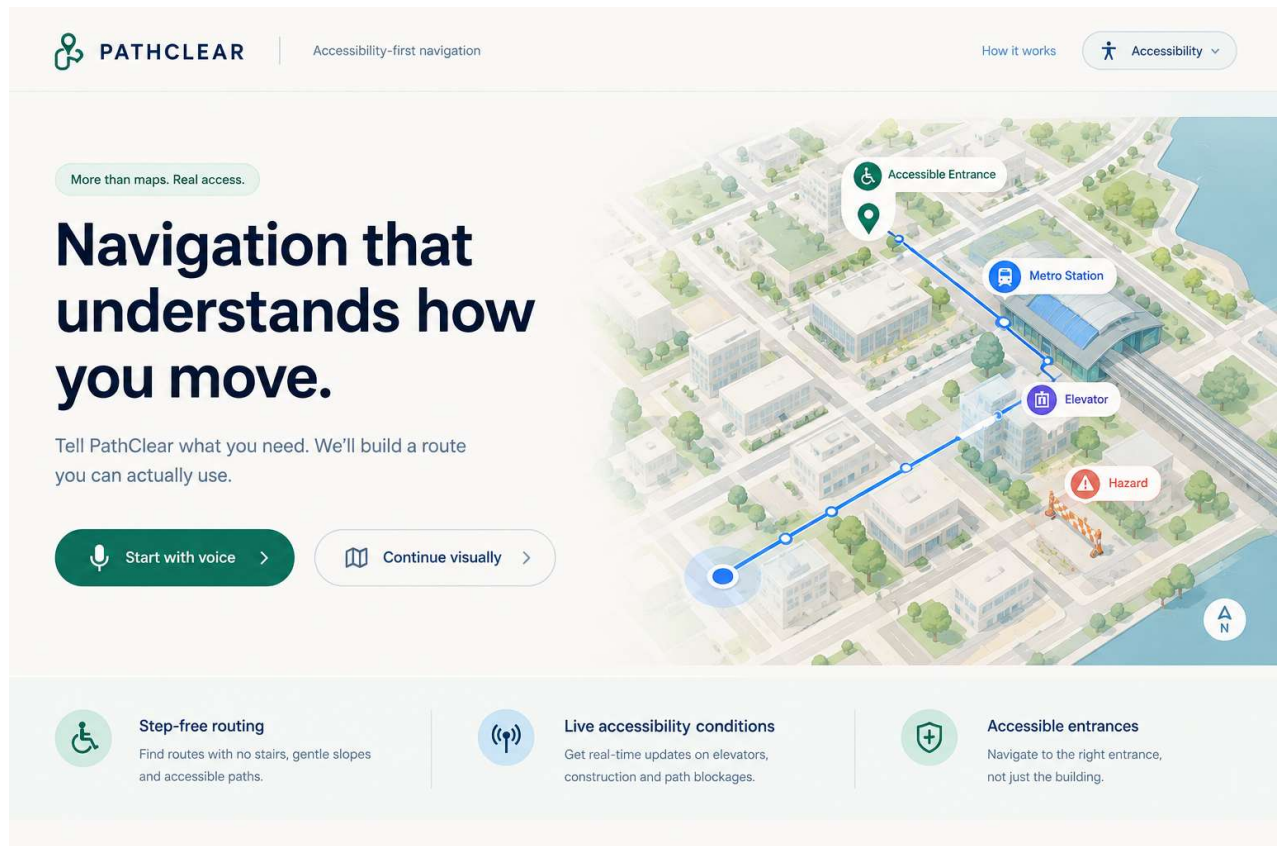
GTFS provides transit stops, routes and schedules. GTFS-Pathways is used conceptually to model the physical journey through a station: street entrance -> accessible gate -> elevator -> concourse -> pathway -> platform. A broken elevator can therefore invalidate a transfer and force a new multimodal journey rather than being shown as a passive warning.

4. Frontend product and page-by-page experience

Visual direction: light theme, warm off-white background, deep navy text, restrained teal/green accessibility accent, clean typography, large controls, minimal decoration, and a map-first product feel. The UI should look like a serious mobility product, not a glowing AI demo.

Page 1 - Welcome and accessibility onboarding

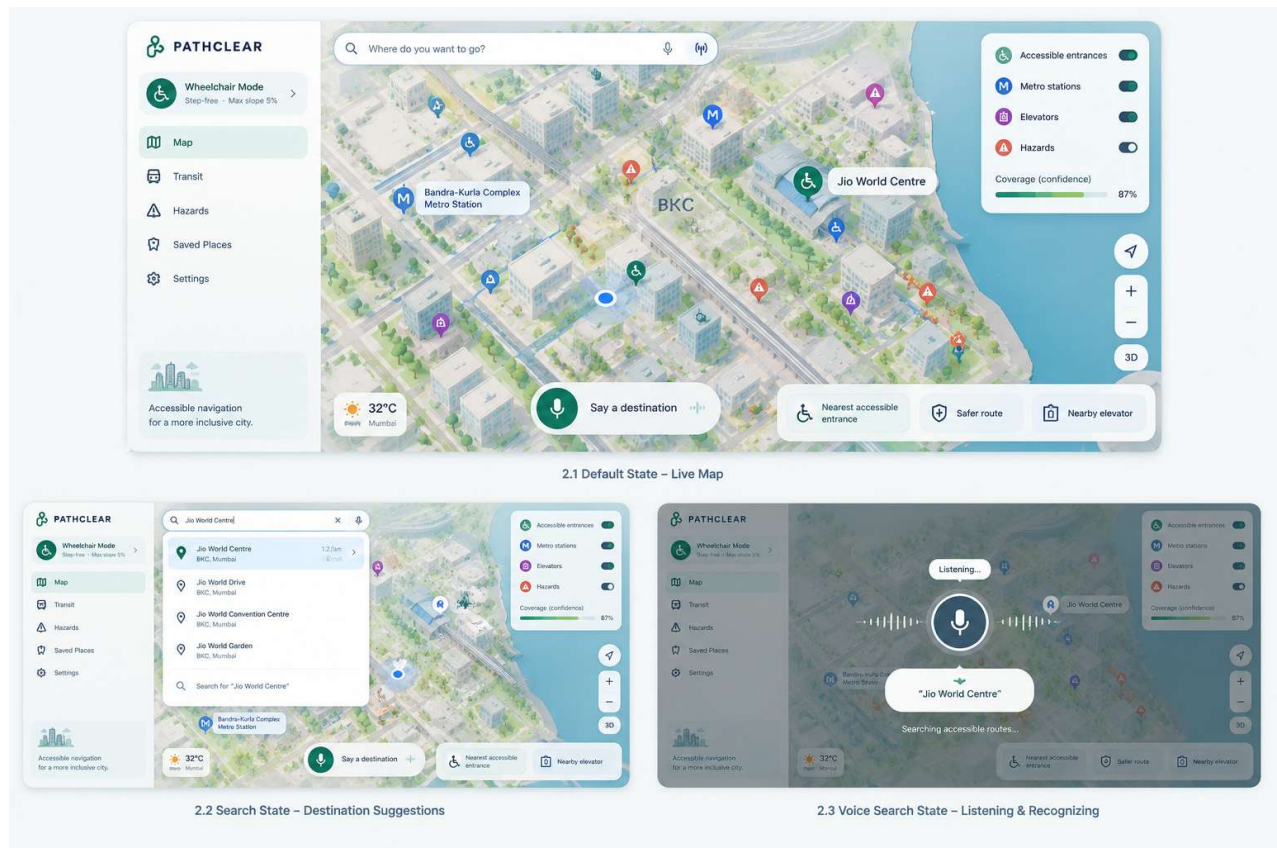
- Hero: "Navigation that understands how you move."
- Primary paths: Start with voice / Continue visually.
- States: default, voice onboarding, visual accessibility selection, detailed mobility constraint setup.
- No prominent "AI-powered" language and no chatbot/orb aesthetic.



Page 1 visual direction - light, calm, accessibility-first.

Page 2 - Main map / destination search

- MapLibre map with current location, subtle 3D buildings and accessibility layers.
- Profile badge such as "Wheelchair Mode - Step-free - Max slope 5%".
- Search bar + voice search.
- Accessibility layers: entrances, elevators, Metro, hazards, confidence/coverage.
- States: default map, search result state, voice-listening state.

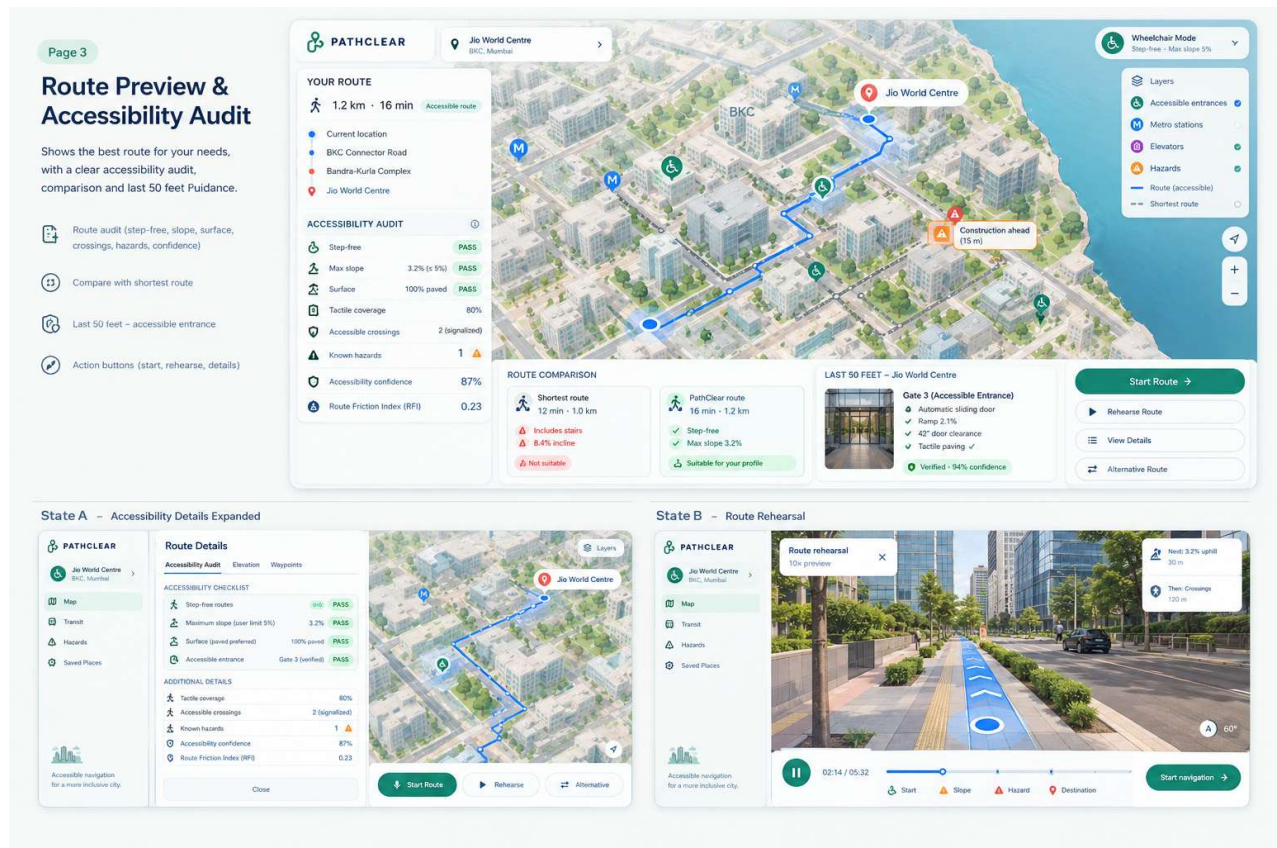


Page 2 with destination search and state variations.

The map shown in the mockups is achievable with MapLibre using vector tiles, 3D fill-extrusions, pitch/bearing, GeoJSON route layers and custom markers. Cesium is not required for the selected product direction.

Page 3 - Route Intelligence / Accessibility Audit

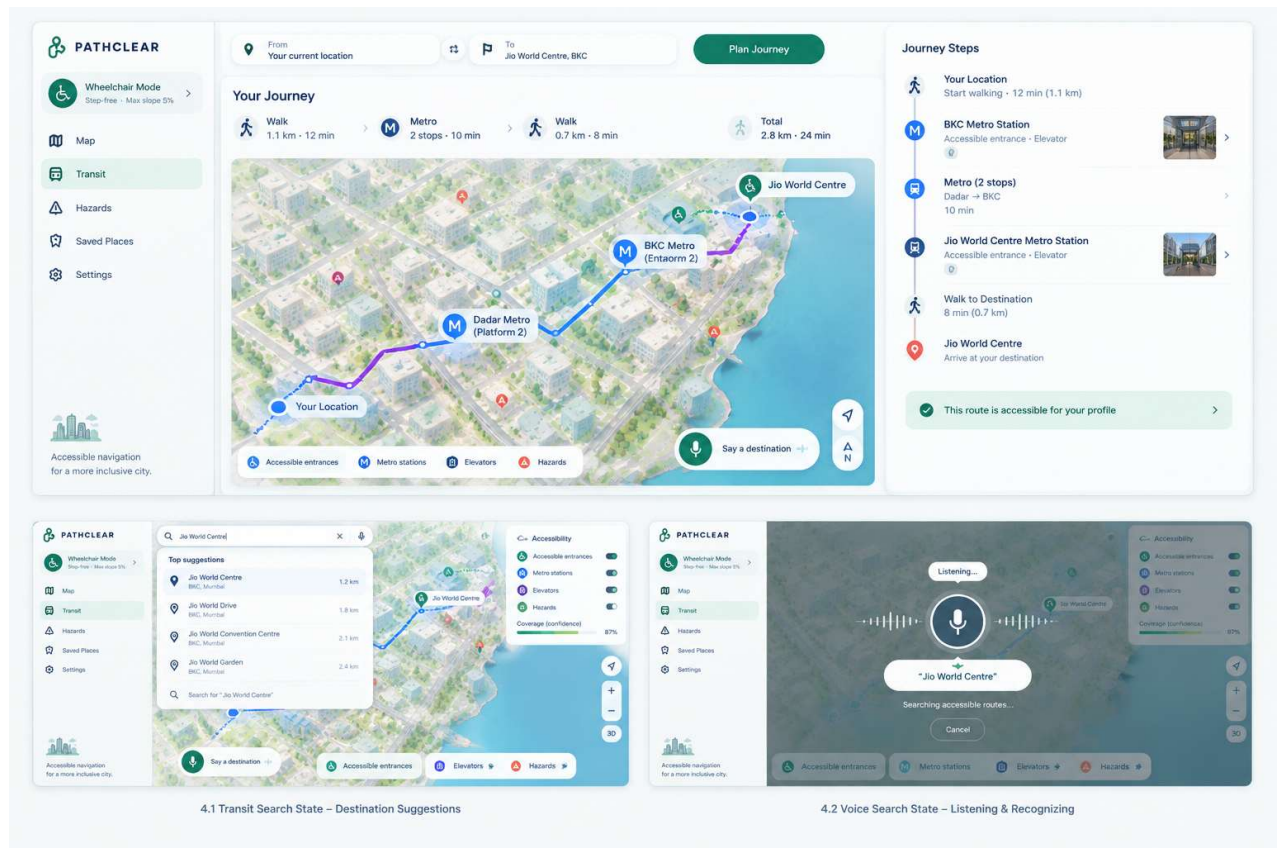
- Show the PathClear route against the normal shortest route.
- Expose why the shortest route fails: stairs, steep slope or other constraints.
- Accessibility Nutrition Label + confidence + RFI.
- Last 50 Feet preview is visible before navigation.
- States: main audit, route details, route rehearsal.
- Route rehearsal is a MapLibre camera animation along route geometry, not photorealistic Street View.



Page 3 route audit, comparison and rehearsal concept.

Page 4 - Multimodal Journey Planner

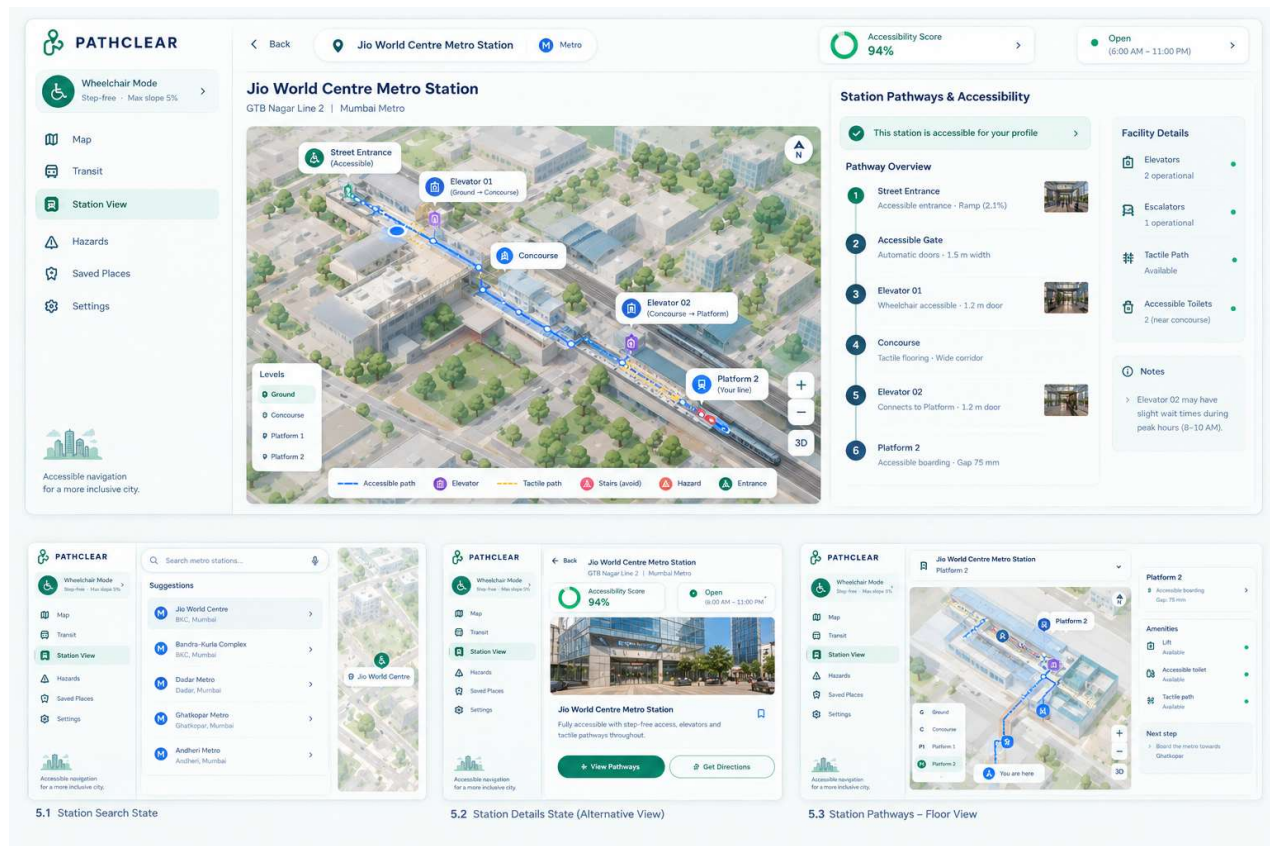
- End-to-end journey timeline across walking + transport + transit.
- Time, estimated cost, walking distance, transfers and accessibility confidence.
- Station-access state and dynamic reroute state when accessibility changes.
- Nearby cab, bike, auto/shared rickshaw and transit options are normalized through provider adapters.



Page 4 multimodal journey planner and dynamic accessibility reroute states.

Page 5 - Station Accessibility / GTFS-Pathways

- Physical transfer graph: entrance -> gate -> elevator -> concourse -> platform.
- Operational elevator and pathway state is visible.
- A failure is a routing event, not only a banner.



Page 5 station accessibility and pathway view.

Pages 6-9

Page	Purpose	Key behavior
6. Active Navigation	Live route guidance	Adaptive UI for wheelchair, blind and deaf users; large maneuvers, voice, haptics, slopes, surface and crossings
7. Hazard Intervention	Reality changed	Construction/blocked path warning, verify still blocked, accessible reroute with explicit trade-off
8. Virtual Cane	Camera perception for blind users	Obstacle position + distance + local action; minimal screen, not an object-detection dashboard
9. Last 50 Feet	Finish at usable entrance	Exact gate, ramp, door, tactile paving, confidence and final guidance

5. Hackathon-winning interactions and judging story

High-impact interactions

- Route comparison: fastest route fails, PathClear route passes.
- Dynamic hazard: construction appears, old route becomes invalid, new step-free route animates in.
- Virtual Cane: camera sees a meaningful path obstacle and gives a directional cue rather than narrating every object.
- GTFS-Pathways: station elevator/pathway failure changes the multimodal journey.
- Last 50 Feet: navigation continues from the building pin to the actual accessible entrance.
- Adaptive experience: same journey becomes voice/haptic-first for blind users and large visual/haptic guidance for deaf users.

Judging story

1. Select a wheelchair profile.
2. Search Jio World Centre.
3. Show shortest route failing and PathClear route passing.
4. Start navigation.
5. Trigger a construction hazard and reroute.
6. Switch to blind mode and demonstrate camera + haptic/voice guidance.
7. Show station pathway accessibility / elevator outage response.
8. Finish with the Last 50 Feet accessible entrance.

Closing message:

"Google Maps answers: how do I get there? PathClear answers: can I actually get there?"

6. Data ingestion, storage, caching and mock strategy

Data categories

Category	Examples	Treatment
Fetch once / batch	OSM network, pedestrian infrastructure, selected elevation, GTFS, selected Mapillary imagery	Normalize and store/import before demo
Dynamic from own DB	hazards, entrances, profiles, station status/pathways, navigation sessions	Read/write through repository/query functions
Computed per request	candidate routes, route audit, RFI, confidence summary, journey ranking	Do not permanently store unless needed
Live device data	GPS, heading, camera frames	Use in active navigation; minimize retention
Mock/seed for hackathon	selected hazards, limited station pathways, Last 50 Feet entrance records, limited provider availability/fare examples when no live provider exists	Seed honestly; do not claim unsupported live integrations

OSM / OSMnx role

OSMnx remains useful for importing and normalizing OpenStreetMap road/pedestrian data even though NetworkX is no longer the final routing engine. The normalized graph is imported into Neo4j. Relevant attributes include sidewalk, surface, incline, steps, ramp, crossing, crossing signals, tactile paving, handrail and width.

DeepLabV3 role

DeepLabV3 was discussed as semantic segmentation for scale-up: identify sidewalk/free-space regions at pixel level rather than only individual objects. It is especially relevant in crowded environments where object-by-object narration becomes useless. It is not part of the first core build; the product contract remains structured traversable-space evidence.

7. Final backend architecture

Frontend: Next.js + React + TypeScript + Tailwind + MapLibre

Backend: FastAPI

- > deterministic services
- > LangGraph agent orchestration
- > Neo4j + GDS graph routing
- > Supabase (Postgres/PostGIS) operational data
- > external integrations / ingestion pipelines
- > vision model API

Device runtime:
GPS + heading + camera -> local navigation/safety controller -> haptic + voice

Responsibility split

Component	Final responsibility
MapLibre	Map rendering, 3D buildings, route layers, hazards, entrances and map interaction
FastAPI	Application API and service orchestration
Neo4j + GDS	Persistent heterogeneous accessibility graph and graph algorithms
Supabase	Operational relational/spatial data, auth if used, hazards/entrances/profiles/sessions
LangGraph	Agent state and tool orchestration
LangChain	Useful model/tool/structured-output utilities under LangGraph, not the main architecture
Vision model	Structured perception; obstacle/entrance evidence
Device runtime	Real-time GPS/heading correction, haptic control, safety loop

8. Neo4j + GDS routing graph

Neo4j replaces NetworkX as the primary graph database/routing layer because the final product needs a persistent heterogeneous graph spanning sidewalks, crossings, entrances, station components, elevators, platforms and hazards. OSM/OSMnx remains an ingestion source/normalizer.

```
(:Sidewalk)-[:CONNECTS_TO]->(:Sidewalk)
(:Sidewalk)-[:HAS_CROSSING]->(:Crossing)
(:Sidewalk)-[:LEADS_TO]->(:Entrance)
(:Station)-[:HAS_ENTRANCE]->(:Entrance)
(:Station)-[:HAS_ELEVATOR]->(:Elevator)
(:Elevator)-[:CONNECTS]->(:Platform)
```

Relationship properties can contain distance, slope, surface, stairs, tactile, width, hazard status and confidence. Neo4j GDS is used for A*/Dijkstra and k-shortest alternatives such as Yen. Candidate paths are then audited and ranked against the profile.

What not to do

- Do not let an LLM invent graph edges, coordinates, slopes or route geometry.
- Do not put Cypher inside API routes, agent nodes or large service functions.
- Do not keep both NetworkX and Neo4j just for the sake of using both.

9. Supabase role and repository pattern

Supabase is the operational database layer. It is still PostgreSQL underneath, with PostGIS available for spatial records. Neo4j handles graph relationships and routing; Supabase handles user/profile data, hazards, entrance metadata, station metadata, navigation sessions and other relational/spatial records.

Supabase repositories

```
db/supabase/repositories/  
user_repository.py  
profile_repository.py  
hazard_repository.py  
entrance_repository.py  
station_repository.py  
navigation_repository.py
```

A repository performs persistence operations only. It does not decide business behavior. For example, `hazard_repository.py` can contain `get_hazard_by_id()`, `get_active_hazards_near_route()`, `create_hazard()`, `update_hazard_confidence()` and `mark_hazard_resolved()`. The hazard service decides when and why to call those functions.

Strict database rule

Every DB read or write must be a separate named function. No inline SQL, Supabase query chain or Cypher inside routes, services, agents or pipelines.

10. Agentic system with LangGraph

PathClear is not "agentic everywhere". The navigation core stays deterministic. LangGraph sits above the tools to interpret intent, choose capabilities, coordinate multi-step workflows and explain verified results.

```
User voice/text
-> classify intent
-> load context/profile
-> planner
    -> route tool
    -> hazard tool
    -> entrance tool
    -> station/pathway tool
    -> transport tool
    -> RAG tool
    -> vision tool
-> deterministic safety validation
-> action / explanation
```

Agent tools

- search_destination()
- get_user_profile()
- calculate_route()
- get_route_audit()
- check_alternative_routes()
- find_nearby_hazards()
- find_accessible_entrance()
- get_station_pathways()
- search_accessibility_knowledge()
- analyze_camera_image()
- reroute()

The agent should never directly calculate GPS, slope, distance, route geometry or hard accessibility validity. It calls deterministic services and receives structured outputs. A safety validator can reject any proposed action that violates hard user constraints.

11. Hybrid GraphRAG / vector RAG

RAG is for unstructured accessibility knowledge, not for route calculation. Structured facts such as ramp slope or elevator status should be queried directly. Documents, reports, venue accessibility text, observations or long descriptions can be indexed for semantic retrieval.

```
Query
-> exact/full-text search
-> vector search
-> merge/rank results
-> graph expansion to real entity
-> grounded context
-> LLM explanation
```

Neo4j can hold graph entities, document/chunk nodes, vector indexes and full-text indexes. Retrieved chunks are linked to the entrance/station/hazard they describe, so semantic evidence can be expanded into structured graph facts. This avoids adding another vector database unless a concrete need appears.

Clear boundary

- GDS decides how to traverse the graph.
- RAG retrieves supporting knowledge.
- LangGraph decides which capability to invoke.
- The LLM explains verified results; it does not invent them.

12. Vision, blind navigation and haptics

Vision pipeline

```
Camera frame
-> vision model
-> structured obstacle / free-space evidence
-> spatial filtering
+ route + GPS + device heading
-> local navigation/safety controller
-> LEFT / RIGHT / STRAIGHT / STOP / HAZARD
-> haptic pattern + concise voice
```

The vision model should return structured information such as object type, relative position, approximate distance, path relevance and confidence. It should not directly control haptics or invent a route.

Haptic design

- A phone generally does not have separate left/right actuators, so direction is encoded with distinct temporal patterns.
- Voice carries semantic meaning; haptics carry immediate spatial correction.
- Examples: left = two short pulses, right = one longer pulse, stop = three strong pulses, hazard = rapid warning pattern.
- The exact real-time haptic controller should live on the device, not behind a FastAPI round trip.

Micro-maneuver vs reroute

If a small obstacle leaves enough clearance, PathClear gives a local correction such as "move slightly right". If the path is fully blocked, it stops and requests a macro reroute. This avoids unnecessary route recalculation and avoids narrating every detected object.

13. Backend folder structure and coding rules

```
backend/
├── app/
│   ├── main.py
│   ├── core/
│   ├── api/v1/routes/
│   ├── schemas/
│   ├── models/
│   ├── db/
│   │   ├── supabase/
│   │   │   ├── client.py
│   │   │   └── repositories/
│   │   │       ├── user_repository.py
│   │   │       ├── profile_repository.py
│   │   │       ├── hazard_repository.py
│   │   │       ├── entrance_repository.py
│   │   │       ├── station_repository.py
│   │   │       └── navigation_repository.py
│   │   ├── neo4j/
│   │   │   ├── client.py
│   │   │   └── queries/
│   │   │       ├── routing_queries.py
│   │   │       ├── accessibility_queries.py
│   │   │       ├── station_queries.py
│   │   │       ├── entrance_queries.py
│   │   │       ├── hazard_queries.py
│   │   │       └── rag_queries.py
│   ├── services/
│   │   ├── routing/
│   │   ├── navigation/
│   │   ├── accessibility/
│   │   ├── hazards/
│   │   ├── entrances/
│   │   ├── transit/
│   │   ├── transport/
│   │   ├── vision/
│   │   ├── geocoding/
│   │   └── rag/
│   ├── agents/navigation/
│   ├── integrations/
│   ├── pipelines/
│   ├── utils/
│   └── constants/
├── scripts/
├── tests/
└── data/
```

Layer rules

Layer	Rule
API routes	Thin HTTP wrappers only: validate -> call service -> return response
Services	Business logic and orchestration; may call repositories, Neo4j query functions and integrations
Repositories / Neo4j queries	One clear DB operation per function; no business decisions
Integrations	External API clients and normalizers only
Pipelines	Batch/background ingestion orchestration
Agent tools	Wrap existing services; never duplicate service logic
Pure logic modules	Weighting, scoring, RFI, constraints, geometry; preferably no DB access

Service function style

```
async def create_accessible_route(request):
    origin = await resolve_origin(request.origin)
    destination = await resolve_destination(request.destination)
```

```
candidates = await generate_candidate_routes(origin, destination, request.profile)
audited = await audit_candidate_routes(candidates, request.profile)
best = select_best_route(audited, request.profile)
entrance = await find_destination_entrance(destination, request.profile)
return build_route_response(best, entrance)
```

The top-level service reads like a business process. Each database operation remains a separate repository/query function.

14. Core APIs, data contracts and tables

Core route endpoint

```
POST /api/v1/navigation/route

Request:
{
  "origin": {"lat": 19.065, "lng": 72.865},
  "destination": {"lat": 19.060, "lng": 72.870},
  "profile": {"mode": "wheelchair", "step_free": true, "max_slope": 5}
}

Response:
{
  "geometry": {...},
  "distance_m": 1200,
  "duration_min": 16,
  "audit": {
    "step_free": true,
    "max_slope": 3.2,
    "surface": "paved",
    "tactile_coverage": 80,
    "accessible_crossings": 2,
    "known_hazards": 1,
    "confidence": 0.87,
    "rfi": 0.23
  }
}
```

Core API surface

- POST /navigation/route
- POST /navigation/reroute
- POST /navigation/voice
- POST /vision/analyze
- GET /hazards/nearby
- POST /hazards/report
- POST /hazards/{id}/verify
- GET /entrances/nearby
- GET /stations/{id}
- GET /stations/{id}/pathways
- POST /agent/invoke

Key Supabase tables

Table	Key fields
users	id, email/name if needed, timestamps
accessibility_profiles	user_id, mobility/vision/hearing constraints, max_slope, step_free, preferences
hazards	type, geometry/lat/lng, severity, confidence, source, status, reported/verified/expires timestamps
entrances	building_id, geometry, step_free, ramp, ramp_slope, door_type/width, tactile, photo, confidence
stations	external_id, name, geometry, operator, accessibility status, confidence
navigation_sessions	user_id, origin, destination, route_id, started/completed/status

15. Two-day build order and demo flow

Build order

Phase	Deliverable
1. Foundation	FastAPI + Neo4j + Supabase clients, schemas, MapLibre shell
2. Ingestion	Import limited BKC OSM/pedestrian graph into Neo4j; seed key accessibility properties
3. Routing	A*/Dijkstra + accessibility weighting + candidate routes
4. Route Audit	Step-free, slope, surface, tactile/crossing, hazards, confidence, RFI
5. Frontend	Page 2 map -> Page 3 audit working from real backend data
6. Hazard reroute	Seed hazard + verify + graph penalty/block + new route
7. Transit	Limited station/pathway graph + elevator outage reroute
8. Multimodal	Normalized transport options + time/cost/accessibility ranking
9. Vision	Camera -> structured obstacle -> local direction demo
10. Last 50 Feet	Seed real-looking entrance data + final guidance
11. Agent	LangGraph tools over already-working services
12. Polish	Adaptive blind/deaf UI, voice, rehearsal, animations, demo hardening

Demo flow

Profile -> destination -> accessibility route comparison -> start route -> construction hazard -> accessible reroute -> blind camera/haptic assist -> station pathway/elevator state -> multimodal transfer -> Last 50 Feet -> accessible door

16. Non-negotiable architecture rules

- Accessibility is part of routing, not a filter after routing.
- Neo4j + GDS is the final primary routing graph; OSM/OSMnx is ingestion/normalization.
- Supabase is the operational relational/spatial store.
- Every DB read/write is its own repository/query function.
- No inline SQL/Supabase/Cypher inside API routes, services, agents or pipelines.
- LangGraph orchestrates deterministic tools; it does not replace routing.
- RAG retrieves knowledge; GDS calculates paths.
- Vision returns structured perception; the local safety controller decides haptics.
- Real-time haptic guidance runs on-device, not through an LLM/network loop.
- Unknown accessibility data remains unknown and lowers confidence.
- Mocked/seeded hackathon data must be clearly described as seeded; never claim a live integration that is not implemented.
- Do not add CrewAI, a second vector database, Redis/Celery/Kafka, microservices or other infrastructure unless a real feature requires it.
- The product should remain a navigation product first and an AI system second.

Final system goal: understand the person, understand the city, plan a feasible journey, react to reality, and finish at the entrance the person can actually use.